UNAB-Ambiental

Informe técnico — Laboratorio 3: MQTT hacia Azure IoT Central

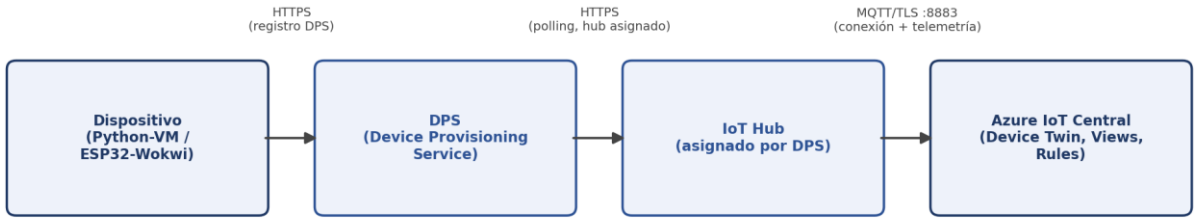
IoT + Cloud + Sistemas Distribuidos · 2026

Integrantes: Andi Lin – U00180153 · Juan Sebastián Buitrago Caballero – U00179631 · Miguel Ángel Solano Díaz – U00177983

Objetivos

Explicar el protocolo real que usa un dispositivo al hablar con IoT Central (MQTT sobre TLS, DPS y luego el hub). Publicar las mismas 3 variables del Laboratorio 2 por dos caminos en la VM: el SDK azure-iot-device y un cliente MQTT explícito con paho-mqtt. Mantener el ESP32 de Wokwi enviando por MQTT. Medir payload, intervalo, latencia y comportamiento ante corte de red. Documentar qué QoS acepta IoT Central.

Diagrama del flujo MQTT → DPS/Hub → IoT Central



El dispositivo se registra vía DPS por HTTPS, recibe el hub asignado, y luego toda la telemetría viaja por MQTT sobre TLS (puerto 8883) directamente hacia ese IoT Hub, que IoT Central expone como Device Twin, Views y Rules.

Etapa 1 — Transporte confirmado

Se confirmó en los logs de ambos nodos (Python-VM y ESP32-Wokwi) el mismo hostname asignado (iotc-4dd6ac2c-bd72-45f1-b37f-5505919e958e.azure-devices.net), puerto 8883, protocolo MQTT sobre TLS, e intervalo de 10 segundos. En Data Explorer se confirmó que el JSON de telemetría (humidity, illuminance, temperature) coincide exactamente con las capabilities del Device Template.

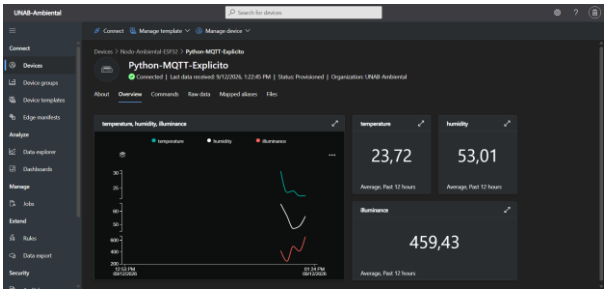
Tabla de mediciones (Etapa 3)

Métrica	Resultado observado
Tamaño del payload	110 bytes constantes (JSON con temperature/humidity/illuminance a 1 decimal + timestamp ISO 8601)
Intervalo configurado vs. observado	Configurado: 10 s. Observado (delta entre timestamps consecutivos): ~10.19 s — el overhead coincide con el tiempo de ACK de QoS 1
Tiempo hasta verse en Central	Prácticamente inmediato tras el ACK del broker (mismo orden que el tiempo de ACK, ~185-200 ms + refresco de Data Explorer)
QoS 0	ACK ~1 ms — no espera confirmación del broker (fire-and-forget)
QoS 1	ACK ~185-200 ms — espera el PUBACK real de IoT Hub
QoS 2	ACK ~184-330 ms, sin cierre de conexión. La documentación de Microsoft indica que IoT Hub no soporta QoS 2 y debería rechazar/cerrar la sesión; no se observó ese comportamiento estricto, lo que sugiere una degradación silenciosa a QoS 1 por parte del broker, o que paho-mqtt completa el handshake local sin que el broker lo procese como QoS 2 real
Corte de red 10-20 s — Python (MQTT explícito)	NO se recupera solo: se desconecta por Keep alive timeout y el siguiente publish() lanza RuntimeError, terminando el programa (ver evidencia)
Corte de red — Wokwi	Pausar la simulación detiene el ESP32 completo sin generar error, pero no equivale a un corte de red real (no hay reloj corriendo para detectar el timeout)

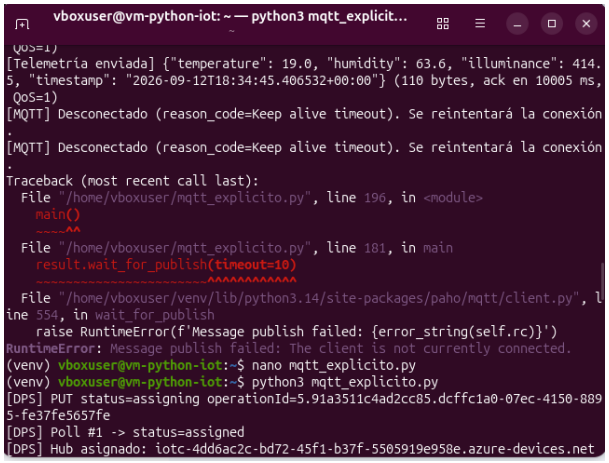
Capturas clave



Data Explorer: telemetría de Python-VM y ESP32-Wokwi, mismo hub



Python-MQTT-Explicito: Connected con datos llegando



Crash del cliente MQTT explícito tras el corte de red (sin reconexión automática)

Tabla comparativa: SDK vs MQTT explícito vs Wokwi

Criterio	SDK Python	MQTT explícito	Wokwi Arduino
Facilidad de implementación	Alta: DPS y SAS resueltos internamente	Media-baja: SAS, DPS por HTTPS y topics a mano	Baja: sin SDK vigente, todo reimplementado con mbedtls en C++
Control del protocolo	Ninguno: topics, QoS y tokens ocultos	Total: QoS por mensaje, topic exacto, token visible	Total, mismo nivel que MQTT explícito
Payload	110 bytes (mismo JSON)	110 bytes (idéntico)	Esquema equivalente de 3 variables
Latencia percibida	No medible directamente (SDK no expone el ACK)	Medida explícita: ~1 ms (QoS 0) a ~190 ms (QoS 1)	Comparable a QoS 1 al usar el mismo patrón MQTT/TLS
Reconexión ante corte de red	Automática e implícita (nunca falló en el Lab 2)	Ninguna por defecto: el script crasheó (RuntimeError); requiere implementación propia	No probado con corte real (solo pausa de simulación completa)
Utilidad para UNAB-Ambiental	Ideal para producción confiable sin equipo dedicado a mantenerlo	Útil para optimización fina (ej. elegir QoS 0 para ahorrar datos móviles)	Única opción si el escenario exige hardware físico real en un aula

Conclusión sobre QoS

IoT Central/IoT Hub acepta QoS 0 y QoS 1 de forma clara y documentada. QoS 2 no está soportado según la documentación oficial, que indica que el broker debería cerrar la conexión ante un publish con QoS 2; en la práctica no se observó ese rechazo — el mensaje se confirmó con un ACK similar al de QoS 1, sugiriendo una degradación silenciosa en vez de un error explícito. Para el escenario UNAB-Ambiental, QoS 1 es la opción más razonable por defecto: garantiza al menos una entrega sin la complejidad de deduplicación que exigiría QoS 2 (que de todas formas no está soportado).

Conclusión: cuándo SDK y cuándo MQTT explícito

Se usaría el SDK para el escenario real de UNAB-Ambiental en producción: la reconexión automática importa más que el control fino del protocolo cuando se trata de decenas de salones sin supervisión técnica constante — el crash observado con MQTT explícito ante un corte de red (RuntimeError sin manejo de excepciones) sería inaceptable en un despliegue real sin trabajo adicional de resiliencia. Se usaría MQTT explícito solo para investigación u optimización puntual, por ejemplo decidir deliberadamente QoS 0 para telemetría no crítica y ahorrar ancho de banda en un despliegue con conectividad limitada, algo que el SDK no permite decidir por mensaje.

Repositorio

https://github.com/AndiLinUnab/unab-ambiental-lab3.git